Inverse Ackermann and Path Compression

N-20261108

Some functions grow so slowly that there is no sensible way to watch them grow on a computer. The inverse Ackermann function is one of them. What makes it interesting is that it appears in the complexity of a very ordinary data structure: a collection of rooted trees used to maintain a partition of a set.

The route from a frighteningly fast-growing function to almost constant-time algorithms is not obvious. We will first build a hierarchy of growth rates, then look carefully at path compression in a disjoint-set forest. A complete charging argument will give the bound $O(m \log^* n)$. The sharper inverse-Ackermann theorem requires a more intricate version of the rank analysis; we will identify what needs to be sharpened rather than disguising an incomplete argument as its proof.[1]

§1.1 Beyond exponentials

Repeatedly adding n produces multiplication; repeatedly multiplying n produces powers. Repeating exponentiation gives power towers. The operation of *iteration* makes it possible to keep going.

For a function f, let $f^{\circ k}$ be its k-fold composition, with $f^{\circ 0}(x) = x$. Define

$$A_0(n) = n + 1, \qquad A_{r+1}(n) = A_r^{\circ(n+1)}(1), \qquad r, n \geq 0. \tag{1.1}$$

This is one convention for an Ackermann hierarchy. Other books shift the indices or change the initial argument.

The first few functions can be calculated directly:

$$\begin{aligned} A_0(n) &= n + 1, \\ A_1(n) &= n + 2, \\ A_2(n) &= 2n + 3, \\ A_3(n) &= 2^{n+3} - 3. \end{aligned}$$

For example, $A_2 = A_1^{\circ(n+1)}(1)$, and every application of A_1 adds two, so $A_2(n) = 1 + 2(n + 1) = 2n + 3$. Applying $x \mapsto 2x + 3$ exactly k times to 1 gives $4 \cdot 2^k - 3$. Taking $k = n + 1$ produces the fourth line.

The next level is already unpleasant. At $n = 1$,

$$\begin{aligned} &A_0(1) = 2, && A_1(1) = 3, \quad A_2(1) = 5, \quad A_3(1) = 13, \\ &A_4(1) = A_3(A_3(1)) = A_3(13) = 2^{16} - 3 = 65533. \end{aligned} \tag{1.2}$$

Going one level higher means

$$A_5(1) = A_4(A_4(1)) = A_4(65533).$$

Even the notation becomes more useful than decimal digits. The function A_4 itself is obtained by iterating a roughly exponential function, $n + 1$ times.

[1] Robert E. Tarjan, *Efficiency of a Good But Not Linear Set Union Algorithm* (1975). The rank-block argument for the iterated-logarithm bound follows the standard exposition in *Introduction to Algorithms*, §21.4, with charging details worked out below.

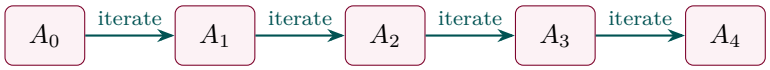


Figure 1.1: Each new Ackermann level outgrows the previous one. Inverting the hierarchy asks how many such levels are needed to reach a threshold.

For definiteness, put

$$\alpha(n) = \min\{r \geq 1 : A_r(1) \geq n\}, \qquad n \geq 1. \tag{1.3}$$

With our convention,

$$\alpha(n) = \begin{cases} 1, & n \leq 3, \\ 2, & 3 < n \leq 5, \\ 3, & 5 < n \leq 13, \\ 4, & 13 < n \leq 65533, \\ 5, & 65533 < n \leq A_5(1). \end{cases}$$

The last range extends to an extraordinary bound. The function is unbounded in the mathematical sense, yet increasing its value by one may require a number far larger than any quantity we can physically store.

In sharp data-structure theorems, the notation $\alpha(m, n)$ often denotes a *two-parameter* inverse Ackermann variant, measuring m operations on n elements. Different normalizations can produce different small numerical values. A claim about $\alpha(n)$ being less than four, for instance, must be read with its particular definition in mind. We use (1.3) when computing values and use Tarjan's conventional asymptotic notation for the sharp complexity result.

§1.2 A forest that keeps changing shape

A disjoint-set union structure maintains a partition into nonoverlapping sets. It supports MAKESET(x), creating a singleton; FIND(x), returning the representative of the set containing x; and UNION(x, y), merging the sets containing x and y.

Represent every set by a rooted tree. Each node x has a parent pointer $p[x]$, and a root satisfies $p[x] = x$. A find follows pointers to the root.

If every union attaches an arbitrary root below another, a tree can become a chain of length $n-1$. A find can then traverse $\Theta(n)$ pointers. The first improvement is to choose the direction of attachment carefully.

Union by rank stores a nonnegative integer rank$[x]$. Initially every rank is zero. When combining distinct roots r, s, attach the lower-rank root below the higher-rank root. If the ranks are equal, attach either below the other and increase the new root's rank by one. Rank is maintained as a historical bound, not as the current depth after compression.

```
MAKESET(x):
   parent[x] = x; rank[x] = 0
FIND(x):
   if parent[x] != x:
      parent[x] = FIND(parent[x])
   return parent[x]
UNION(x, y):
   a = FIND(x); b = FIND(y)
   if a == b: return
   if rank[a] < rank[b]: swap(a, b)
   parent[b] = a
   if rank[a] == rank[b]: rank[a] += 1
```

The recursive call implements *full path compression*. Once the root is known, all visited vertices on that path become its direct children.

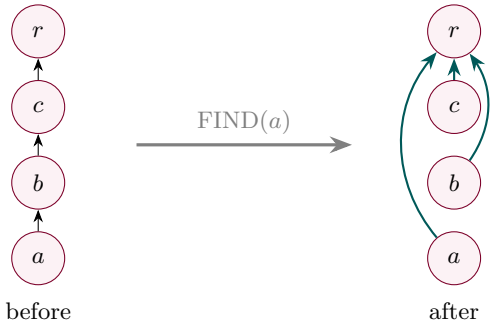


Figure 1.2: One find traverses a chain and redirects all nonroot nodes to the common root. A later find cannot keep paying for the same chain unless subsequent unions change the forest.

§1.3 Rank is an accounting device

Two elementary lemmas explain why rank is so useful.

Lemma 1.3.1 (Strict ranks on every parent chain)

For a nonroot x,

$$\operatorname{rank}[x] < \operatorname{rank}[p[x]].$$

Ranks never decrease. Once a node becomes a nonroot, its rank never changes.

Proof. The claims hold when every node is a singleton root. A link attaches a lower-rank root below a higher-rank root, or attaches equal-rank roots and then increases the new root's rank; the strict inequality holds in either case. Ranks change only for roots involved in ties. Path compression replaces a parent by an ancestor, whose rank is strictly larger than the replaced parent's rank unless the parent was already the root. □

Lemma 1.3.2 (Exponential size)

Whenever x is a root,

$$|\operatorname{tree}(x)| \geq 2^{\operatorname{rank}[x]}.$$

Consequently every rank is at most $\lfloor \log_2 n \rfloor$ if a total of n elements have been created.

Proof. A singleton has size $1 = 2^0$. If two roots have different ranks, the larger-rank root keeps its rank and gains nodes, so the bound remains true. If both have rank r, their disjoint trees each have at least 2^r nodes; the new root has rank $r+1$ and at least 2^{r+1} nodes. Path compression changes no set sizes. Induction proves the claim. □

A stronger counting statement holds over an entire operation sequence.

Lemma 1.3.3 (Ranks are rare)

Among n created elements, at most $n/2^r$ are ever assigned rank r, for every integer $r \geq 0$.

Proof. Whenever a root first receives rank r, mark every element in its then-current tree with that root's name. The tree has at least 2^r elements. Different roots first attaining rank r have disjoint marked groups: once the first such root is attached to another, all its future ancestors have rank greater than r. None of its elements can later belong to a different root that *first* attains rank r. Therefore no element is marked twice at the same rank. Since there are n elements, there can be at most $n/2^r$ such groups. Every node first acquires a rank while it is a root, so this counts all nodes ever assigned that rank. □

Here are two valuable resources for the analysis: rank increases strictly as we climb a parent chain, and very few nodes can carry a high rank.

§1.4 Why one find can still be logarithmic

Even union by rank and full path compression do not make *every individual find* constant time. Before any compressing find is performed, repeatedly combine trees of equal rank. Two singletons form a tree of rank 1; two rank-1 trees form one of rank 2, and so on. After building a set of 2^r elements this way, a leaf can lie at depth r. Its first find takes $\Theta(r) = \Theta(\log n)$ time.

The point of path compression is that the long find modifies the structure so that subsequent finds on the same path become cheap. The interesting upper bound is therefore *amortized*: it controls the total cost of a long sequence of operations, not the cost of every operation separately.

It is useful to recall the weaker logarithmic bound first. By the strict-rank lemma, a parent chain has at most $\lfloor \log_2 n \rfloor + 1$ distinct ranks, so every find costs $O(\log n)$. The analysis we really want must count how often a node can make us pay for the *same portion* of that rank range.

§1.5 Grouping ranks into towers

Define the iterated binary logarithm by

$$\log_2^* x = \min\{k \geq 0 : \underbrace{\log_2 \circ \cdots \circ \log_2}_{k \text{ times}}(x) \leq 1\}.$$

For example,

$$65536 \longmapsto 16 \longmapsto 4 \longmapsto 2 \longmapsto 1, \qquad \log_2^*(65536) = 4.$$

It is much slower growing than $\log_2 n$, but still grows faster than an inverse Ackermann function.

Construct tower boundaries

$$B_{-1} = -1, \quad B_0 = 1, \qquad B_{j+1} = 2^{B_j}. \tag{1.4}$$

Thus

$$B_0 = 1, \quad B_1 = 2, \quad B_2 = 4, \quad B_3 = 16, \quad B_4 = 65536, \ldots.$$

We partition the possible ranks into consecutive groups

$$G_j = \{B_{j-1} + 1, \ldots, B_j\}, \qquad j \geq 0.$$

The first groups are

$$G_0 = \{0, 1\}, \quad G_1 = \{2\}, \quad G_2 = \{3, 4\}, \quad G_3 = \{5, \ldots, 16\}.$$

Only those groups intersecting the range $0, \ldots, \lfloor \log_2 n \rfloor$ can occur. Their number is

$$b = O(1 + \log_2^* n).$$

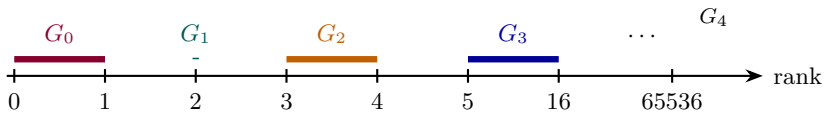


Figure 1.3: Successive rank groups end at tower values $1, 2, 4, 16, 65536, \ldots$. The geometric spacing here is schematic, not numerical: the last group is enormously wider than it looks.

Why choose these apparently absurd group widths? Because the rank-counting lemma makes the high-rank groups correspondingly rare. The rapid growth of B_j will cancel the scarcity of nodes having ranks in G_j.

§1.6 A complete iterated-logarithm bound

Consider a sequence of m operations after at most n MAKESET operations, with $m \geq n$. We count the parent pointers traversed by every FIND. A UNION uses two FINDs and one constant-time link, so replacing unions by those primitive actions changes the operation count by at most a constant factor.

Take one FIND path

$$x_0 \longrightarrow x_1 \longrightarrow \cdots \longrightarrow x_\ell, \qquad x_\ell = \text{root}.$$

The ranks strictly increase. Classify its traversed edges in three ways:

1. An edge $x_i \to x_{i+1}$ that passes from one rank group G_j to a higher group. There can be at most $b-1$ of these per find.

2. The last edge $x_{\ell-1} \to x_\ell$, which we pay for directly if it has not already been counted. This contributes at most one extra edge per find.

3. Every other edge stays inside a single rank group. We charge this edge to its lower endpoint x_i.

Only the third type needs a global argument. Suppose $x = x_i$ has rank in group G_j, and its edge to its parent x_{i+1} is of the third type. Since the parent is *not* the final root, full path compression redirects x all the way to x_ℓ. By strict increase along the original chain,

$$\mathrm{rank}[x_\ell] > \mathrm{rank}[x_{i+1}].$$

The rank of x is now fixed forever, because x is a nonroot; the rank of its parent can only increase over time. On every future occasion when the same type of edge is charged to x, its parent's rank has increased by at least one. As long as that parent's rank stays in G_j, it cannot exceed B_j.

Therefore each nonroot x whose rank lies in G_j receives at most

$$B_j - B_{j-1}$$

charges of the third type during the entire sequence. This may be a huge number for a large j, but it applies to very few nodes.

Let N_j be the number of nodes whose ranks ever belong to G_j. The rank-count lemma gives, for $j \geq 1$,

$$\begin{aligned} N_j &\leq \sum_{r=B_{j-1}+1}^{B_j} \frac{n}{2^r} \\ &< \frac{n}{2^{B_{j-1}}} = \frac{n}{B_j}. \end{aligned}$$

For $j = 0$, the two possible ranks $0, 1$ give $N_0 \leq 3n/2$. Thus the total number of third-type charges from a nonzero group is at most

$$N_j(B_j - B_{j-1}) \leq \frac{n}{B_j} B_j = n,$$

and the group G_0 contributes at most a constant times n. Summing over all at most b groups, there are only $O(nb)$ charged edges of the third type in the *entire sequence*.

The other two edge types contribute $O(mb)$, because they occur at most $O(b)$ times per find. Every traversed edge has been paid for, giving the promised theorem.

Theorem 1.6.1 (A fully proved bound)

With union by rank and full path compression, a sequence of $m \geq n$ MAKESET, UNION and FIND operations on n elements takes

$$\boxed{O(m(1 + \log_2^* n))} \tag{1.5}$$

total time.

The important step is the cancellation

$$\underbrace{\frac{n}{B_j}}_{\text{few nodes}} \cdot \underbrace{B_j}_{\text{many charges per node}} = n.$$

Even though the groups rapidly become huge, each one can account for only $O(n)$ expensive same-group traversals. The number of groups controls the answer.

§1.7 Where the inverse Ackermann function enters

Tarjan's sharper theorem replaces the iterated logarithm by an inverse Ackermann function. In standard forms it states that a sequence of $m \geq n$ operations can be carried out in

$$O(m\,\alpha(m,n)) \quad \text{or, with a one-variable upper bound,} \quad O(m\,\alpha(n)). \tag{1.6}$$

Here α denotes one of the conventional functional inverses of an Ackermann hierarchy. The two-parameter function adjusts its threshold to the ratio m/n; the precise definition affects small values, though not the nearly constant character of the complexity.

Our rank-block proof already explains why this kind of hierarchy appears. A traversed edge is inexpensive if it crosses to a higher rank class; if it stays in one class, compression makes the parent's rank grow, so the edge can be charged only a limited number of times. To improve $\log^* n$ further, a single tier of tower-sized classes is insufficient. The refined analysis assigns several *levels* inside rank intervals, with advancement thresholds defined recursively by iterating the preceding level's threshold function. A hierarchy of repeated exponentials and their iterations naturally appears. The number of levels required to dominate all ranks is then controlled by the inverse Ackermann function. Establishing the sharp bound needs a carefully defined multilevel potential or charging scheme and a proof that the level transitions account for every traversed edge; we do not silently substitute the tower-block estimate for that argument.

There is an asymptotic point worth keeping. The bound (1.6) grows without limit as $n \to \infty$. It is mathematically different from $O(m)$, and in relevant computational models there are matching inverse-Ackermann lower bounds for sufficiently general sequences of unions and finds. Calling the running time constant per operation is an extremely good engineering approximation, while the theorem says something more delicate.

If you construct a rank-r tree by repeatedly merging equal-rank trees, an unlucky first FIND can still cost $\Theta(r) = \Theta(\log n)$. The near-constant guarantee comes from adding costs over the complete sequence. In this sense the inverse Ackermann function measures the remaining cost of a long history of *self-repairing* pointer traversals, not the height of a particular tree.